

Responsible Disclosure – Security Vulnerabilities in BlitzIt

From: Patrick Dundas
pjd9@proton.me

To: team@support.blitzit.app
team@support.blitzit.app

On: Sunday, August 17, 2025 at 1:57:12 PM

BlitzIt Engineering / Security Team,

I am writing to responsibly disclose potential security vulnerabilities I discovered in your application *BlitzIt*, which is hosted at:

- blitzitapp1.firebaseio.com
- blitzitapp1.appspot.com
- blitzitapp1-default-rtdb.europe-west1.firebaseio.com
- integration-prod.blitzit.app

I want to emphasize that I am a security researcher acting in good faith. I have not exploited these issues beyond the minimal steps required to validate their existence, and I have only tested with my own test user accounts.

Vulnerability #1 – Firestore DB Improper Access Control

- **CWE-284: Improper Access Control**
- **CWE-266: Incorrect Privilege Assignment**
- **CWE-732: Incorrect Permission Assignment for Critical Resource**

Due to a misconfiguration in Firestore security rules for the (`default`) database, authenticated users can arbitrarily update fields in their own users document. This includes sensitive subscription and billing metadata such as `account.trialEndsOn`, `account.plan`, `account.stripId`, and `account.brevold`.

Impact: A malicious user could extend their free trial indefinitely or alter billing identifiers, bypassing account restrictions.

Reproduction:

While authenticated as a trial user, run an update against the user's document in the `users` collection to extend a trial indefinitely, or modify [account.plan.id](#) to change their plan type and available features.

```
firebase.collection("users")
  .doc("<myUserId>")
  .update({ "account.trialEndsOn": new Date("3025-09-01T00:00:00Z") });
```

After relaunching the BlitzIt desktop app, the trial countdown reflects thousands of days remaining.

Vulnerability #2 – Firebase Storage Improper Access Control

- **CWE-284: Improper Access Control**
- **CWE-639: Authorization Bypass Through User-Controlled Key**
- **CWE-200: Exposure of Sensitive Information to an Unauthorized Actor**

User-uploaded files in `users/<uid>/...` are publicly retrievable without authentication or a download token.

Impact: An attacker who learns or guesses a file path can access another user's uploads.

Reproduction:

- Upload a profile avatar or list icon in-app.
 - Access the object directly
 - <https://firebasestorage.googleapis.com/v0/b/blitzitapp1.appspot.com/o/users%2F<uid>%2F<filename>.png?alt=media>
 - Removing the appended `?alt=media` returns sensitive metadata, such as a valid download token
-

Recommendations

- Restrict Firestore rules to enforce **field-level access control** (prevent users from editing subscription metadata like `account.trialEndsOn`, `account.plan`, etc.).
- Update Firebase Storage rules to require authenticated access with UID-matching paths or signed download tokens.
- **Establish a dedicated security contact channel:** Publish a `security.txt` file (per [RFC 9116](#)) or similar disclosure instructions on your primary domain, so security researchers know exactly where to send sensitive findings. Relying on general support email addresses is not ideal, as those inboxes are often monitored by customer service staff rather than security engineers, which can delay or mishandle critical vulnerability reports.

I am happy to provide additional details, proof-of-concept requests, or assist in retesting after remediation.

Thank you for your time and commitment to user security.

Sincerely,

Patrick Dundas

Cybersecurity Researcher, Software Engineer

Reach me securely at: pjd9@proton.me

